SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence **COURSE CODE: CSA17**

COURSE OUTCOME COVERED

CO1: Apply AI basics such as intelligent agents and uninformed search methods to solve problems effectively. *(BL3)*

Assessment Tool Weightage: 35%

QUESTIONS: 5

Duration : 1 Hour
Total Marks:50

Annexure A

Analytical Problem Solving

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

1. Water Jug Problem

Problem: Two unmarked jugs are available: one with a capacity of 4 gallons and another with a capacity of 3 gallons. The aim is to obtain exactly 2 gallons in the 4-gallon jug. Water can be filled from a pump, poured between the jugs, or discarded.

Solution

1. Fill the 3-gallon jug completely.

2. Pour the water from the 3-gallon jug into the 4-gallon jug. The 4-gallon jug now contains 3 gallons.

3. Fill the 3-gallon jug again.

4. Pour water from the 3-gallon jug into the 4-gallon jug until the 4-gallon jug is full. Only 1 gallon can be transferred, leaving 2 gallons in the 3-gallon jug.

5. Empty the 4-gallon jug onto the ground.

6. Pour the remaining 2 gallons from the 3-gallon jug into the 4-gallon jug.

Final state: the 4-gallon jug contains exactly 2 gallons of water.

State representation: (x, y), where x is the amount in the 4-gallon jug and y is the amount in the 3-gallon jug. The goal state is (2, 0).

2. Mars Rover as an Intelligent Agent

A Mars Rover is an autonomous robotic agent that observes its surroundings, navigates the Martian surface, collects scientific data, and analyses rock and soil samples.

i. Percepts of the agent

The rover receives percepts through cameras, navigation cameras, spectrometers, environmental sensors, temperature sensors, wheel encoders, and other scientific instruments. These percepts include images of terrain, obstacles, slopes, rock and soil properties, temperature, and instrument readings.

ii. Operating environment

The environment is partially observable because the rover cannot sense everything around it at once. It is stochastic because terrain and equipment behaviour may be uncertain. It is sequential because each action affects later actions. It is dynamic because environmental conditions can change, and it is continuous in location, movement, and time. The rover operates in a single-agent environment, although communication with Earth and other missions may be relevant.

iii. Actions the agent can take

The rover can move forward or backward, turn, stop, capture images, inspect rocks, use scientific instruments, collect or analyse samples where equipped, transmit data to Earth, and enter a safe mode when a fault or hazard is detected.

iv. Performance evaluation

Performance can be evaluated by the scientific value and accuracy of collected data, successful sample analysis, distance safely explored, navigation accuracy, avoidance of hazards, energy efficiency, equipment reliability, and the amount of useful data transmitted within mission constraints.

v. Suitable agent architecture and reason

A model-based, goal-based agent is suitable. A model-based component maintains an internal map and information about the rover's condition when the environment is only partially observable. A goal-based component plans actions to reach scientific targets while avoiding hazards. Utility-based decision-making can also help balance scientific benefit, time, energy, and safety.

3. Eight-Queens Problem: Search Space and Formulation

The objective is to place eight queens on an 8 × 8 chessboard so that no two queens attack one another. Queens attack along rows, columns, and diagonals.

Problem formulation

- **Initial state:** An empty 8 × 8 chessboard.
- **State:** A partial or complete placement of queens. A convenient representation is one queen per column, recorded as the row number for each column.
- **Actions:** Place a queen in a safe square in the next column.
- **Transition model:** The selected square receives a queen, producing a new board configuration.
- **Goal test:** Eight queens are placed and no pair shares a row, column, or diagonal.
- **Path cost:** For a basic constraint-satisfaction formulation, each placement can have unit cost; the main objective is to find a valid solution.

Search strategy

Backtracking search can be used. Place a queen column by column. For each column, try a row that does not conflict with any previously placed queen. If no safe row exists, return to the previous column and move that queen to another row. Continue until all eight queens are placed.

A placement is safe when, for every earlier queen, the row differs and the absolute row difference is not equal to the column difference.

Example valid row-by-column arrangement: [1, 5, 8, 6, 3, 7, 2, 4]. This is one valid solution; each entry gives the row of the queen in the corresponding column.

4. OLA Cab Booking as a Problem-Solving Agent

A goal-based problem-solving agent can be used to help a customer book a cab from a starting location to a destination. It can also use utility-based selection to compare cab types according to fare, travel time, comfort, and availability.

Problem formulation

- **Initial state:** The customer opens the OLA application and provides pickup and destination locations.
- **Goal state:** A suitable cab is selected and the booking is confirmed for the destination.
- **Actions:** Enter locations, view available cab types, compare estimated fare and arrival time, select a cab, and confirm booking.
- **Constraints:** Budget, estimated arrival time, availability, customer comfort, and trip requirements.

Pseudocode

```
START
  Read pickup_location
  Read destination
  Read customer_preferences
  cab_options = GetAvailableCabs(pickup_location, destination)

  IF cab_options is empty THEN
     Display "No cabs available"
     STOP
  END IF

  FOR each cab in cab_options DO
     Estimate fare, pickup_time, and trip_duration
     Check whether cab meets customer_preferences
  END FOR

  suitable_cabs = Filter cab_options using preferences
  Display suitable_cabs with fare and estimated time
  selected_cab = GetCustomerSelection()

  IF selected_cab is available THEN
     booking = ConfirmBooking(selected_cab)
     Display booking_details
  ELSE
```

```
    Display "Please select another available cab"
  END IF
END
```

5. Uniform Cost Search (UCS) for the Delivery Network

Uniform Cost Search expands the frontier node with the lowest cumulative path cost g(n). It is appropriate when route costs are non-negative and the aim is to find the least-cost path.

Given graph edges

From the diagram in the question, the directed edges and costs are:

- S → A = 1
- S → G = 12
- A → B = 3
- A → C = 1
- B → D = 3
- C → D = 1
- C → G = 2
- D → G = 3

UCS cost exploration

Node	Cumulative cost	Path / note
S	0	Start node
A	1	S → A
C	2	S → A → C
D	3	S → A → C → D
G	4	S → A → C → G

Other candidate routes include S → A → B → D → G with cost 1 + 3 + 3 + 3 = 10, and the direct route S → G with cost 12. The route S → A → C → G has cost 1 + 1 + 2 = 4.

UCS pseudocode

```
UCS(start, goal):
  frontier = priority queue containing (cost=0, start)
  best_cost[start] = 0
  parent[start] = NONE

  WHILE frontier is not empty:
    (cost, node) = remove entry with smallest cost
    IF cost != best_cost[node]:
```

```
    CONTINUE
IF node == goal:
    RETURN reconstruct_path(parent, goal), cost

FOR each edge (node, next, edge_cost):
    new_cost = cost + edge_cost
    IF next not in best_cost OR new_cost < best_cost[next]:
        best_cost[next] = new_cost
        parent[next] = node
        insert (new_cost, next) into frontier

RETURN "No path found"
```

Final answer: the least-cost path is S → A → C → G, with a total cost of 4.

Conclusion

These problems demonstrate how artificial intelligence represents states, selects actions, and searches for solutions. The water jug problem uses state transitions, the Mars Rover illustrates intelligent-agent design, the eight-queens problem uses constraint-based search, OLA cab booking uses goal-directed decision-making, and Uniform Cost Search finds a least-cost path.